Labo 3: Partitioning the grid into warps and reducing thread divergence
Geavanceerde computerarchitecturen

Jan Martens & Leon Cherchye

November 16, 2024



Session Date: 8 november, 2024

Class: Ma ELICT optie software-engineering

1 Introduction

In dit labo experimenteerden we met het aantal threads binnen een block, en de invloed hiervan op de execution time. We keken hierbij specifiek naar de invloed van de warp-size was op het aantal threads per block.
Tevens onderzochten we de invloed van thread-divergence op de algehele execution time en zochten we naar een oplossing voor dit probleem.

2 Specificatie van de GPU
Nvidia MX450

De GPU die ik gebruikte, heeft 896 cores en 14 streaming multiprocessors. Dit geeft dus een warp-grootte van 64 cores.

3 Invloed van het aantal threads/block (volledige vs gedeeltelijke warps) op de uitvoeringstijd

We zien heel duidelijk dat gebruik maken van weinig threads, nadelig is. Zolang er minder threads in een block zitten dan er threads in een warp zitten, daalt de uitvoeringstijd sterk. In ons geval zien we dus duidelijk dat de uitvoeringstijd sterk daalt tot alle cores in een warp ook effectief gebruikt worden.
Wanneer er dan meer dan 64 threads gescheduled worden, stijgt de uitvoeringstijd opnieuw significant en daalt deze geleidelijk tot er terug een veelvoud van 64 threads per block gescheduled worden. Dit komt omdat er op dat moment (indien er geen divergence is tussen de threads), elke core evenveel werk uitvoert en constant aan het werk is.
Wat opvalt is dat op de grens tussen 320 threads/block en 322 threads per block, de uitvoeringstijd pas na 192 extra threads/block een nieuw minimum bereikt. Er wordt dus geen minimum bereikt wanneer het aantal threads/block 384 (=64*6) of 448 (=64*7) is. Pas wanneer het aantal threads/block 512 is, wordt een nieuw minimum bereikt in de uitvoeringstijd.
Hierna wordt er schijnbaar een nieuw minimum bereikt wanneer alle threads in 1 block zitten. Hierbij duurt het dus nog langer voor er een nieuw minimum bereikt wordt.
De verklaring voor dit fenomeen laat ons het raden.

4 Invloed van thread-divergence op de uitvoeringstijd

Wanneer we het rode kanaal van de afbeelding op een andere, meer rekenintensieve manier verwerken, zien we in Figure2 dat de uitvoeringstijd licht stijgt. Ook zien we dat er af en toe uitschieters te zien zijn in de uitvoeringstijd. Het algemene verloop is grosso modo identiek aan dat als in het eerste experiment.
De langere uitvoeringstijd is te verklaren doordat de uitvoeringstijd van de berekeningen voor de waarden in het rode kanaal langer duurt dan die voor de waarden in het groene en blauwe kanaal. De threads die deze complexere berekeningen uitvoeren, zullen er dus voor zorgen dat de threads die de eenvoudigere berekeningen doen moeten wachten op de tragere. Alle cores in één warp worden namelijk simultaan gescheduled, dus moeten ze wachten op een nieuwe opdracht tot de volledige warp vrijgegeven wordt.
De uitschieters zijn mogelijks te verklaren door een ongelukkig toeval. Het zou bijvoorbeeld kunnen dat er toevallig heel veel rode waarden door 1 core berekent moesten worden, waardoor 1

core veel langer werk had dan de andere cores in de warp, waardoor deze allemaal sterk vertraagd werden.

4.1 Optimized kernel

Om het probleem van thread-divergence en bijgevolg 'wachtende' cores te omzeilen, kunnen we gebruik maken van twee verschillende kernels. Hierbij worden alle threads die het rode kanaal behandeld, samen gescheduled. Analoog worden de threads voor het groene en blauwe kanaal samen gescheduled. Het resultaat van deze implementatie is te zien in de groene grafiek in Figure2.
We zien onmiddelijk dat de uitvoeringstijd nu constant is. De verklaring hiervoor is ons het raden. Er wordt schijnbaar geen voordeel gehaald uit het schedulen van een veelvoud van 64 threads/block.
Een tweede fenomeen dat opvalt is dat er geen uitschieters meer zijn. Dit lijkt ons logisch, aangezien er nu normaliter geen asymmetrisch belaste cores zijn per block.

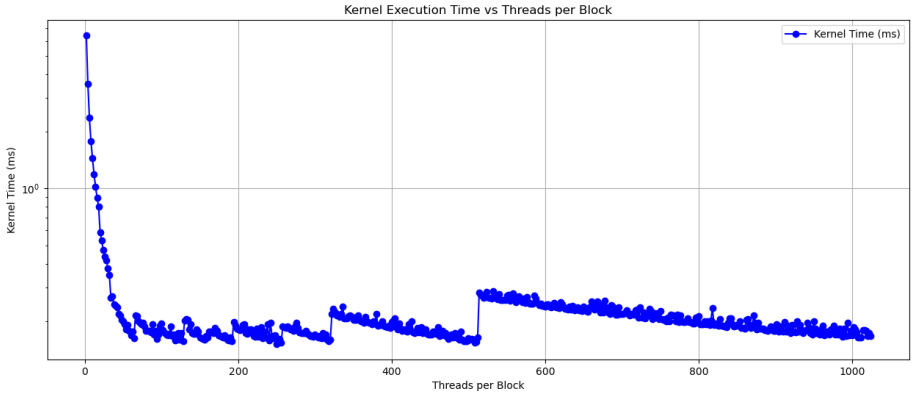


Figure 1: Kernel execution time vs threads per block

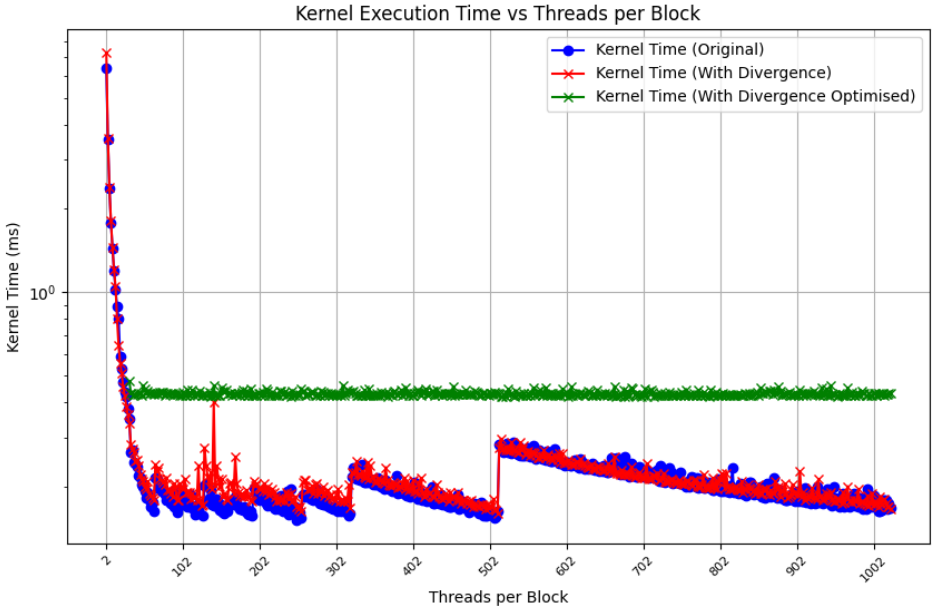


Figure 2: Kernel Execution Time vs Threads per Block Comparison